

One-to-one SkewGRAM: from biology to NLP

Bechir Mady¹, Abderahmane Abderrahmane¹, El Moustapha Mohamed El Moustapha¹, and Mohamedou Yahya Cheikh Mohamed Vall¹

Institut Supérieur du Numérique (SupNum), Nouakchott, Mauritania
{22014, 22053, 22013, 24239}@supnum.mr

Abstract. The One-to-one SkewGRAM problem is an NP-hard combinatorial optimization challenge from bioinformatics. It aims to find the longest path in a directed acyclic graph D such that its vertices induce a connected subgraph in an undirected graph G . Exact methods (ILP) are exponentially slow, and Graph Neural Network (GNN) heuristics generate trivially short paths (≈ 1.2 nodes). We propose **One-to-one Skew-GRAM**, a Skip-Gram-style node embedding model (DeepWalk/node2vec paradigm) trained on random walks over D with structural negative sampling, coupled with a guided decoder that combines three signals at each step: (i) an exact dynamic-programming lookahead on the longest remaining path in the DAG D , which prevents the walk from stalling in dead ends, (ii) the learned embedding similarity, and (iii) a bonus rewarding successors that preserve connectivity in G . A final exact step extracts, via sliding-window Union-Find, the longest contiguous sub-path that is strictly connected in G . Evaluated on 38 synthetic instances (100 nodes, 36 with a known ILP2 optimum), our approach solves each instance in **2.7 seconds** on average (i.e., $\approx 10\times$ faster than the exact ILP2 solver), while discovering paths of **7.13 nodes** on average (mean gap of -30% to the ILP2 optimum, and matching or beating it on 13/36 instances), vastly outperforming the GNN state-of-the-art and offering a strong quality-time tradeoff for genomic-scale scaling.

Keywords: Combinatorial Optimization · Bioinformatics · Graph Algorithms · Node Embeddings · Skip-Gram · Dynamic Programming · Connected Components · Union-Find.

1 Introduction

Artificial Intelligence (AI) and machine learning are experiencing growing adoption in bioinformatics. In developing countries, adopting AI-based approaches offers unique potential to bypass traditional development stages and accelerate biological discoveries, notably in disease treatment and agriculture. A major challenge is the integration and analysis of multiple layers of biological data (multi-omics).

Living cells are governed by multiple networks of interactions. Cellular metabolism can be modeled as a directed graph, where metabolites are connected by directed biochemical reactions illustrating a causal flow. Simultaneously, genomic proximity or protein-protein interactions (PPI) can be modeled by undirected graphs,

describing strong functional associations. Identifying common sub-structures in these two graphs allows for the discovery of essential functional modules conserved throughout evolution.

The **One-to-one SkewGRAM** problem formalizes this challenge: find a directed path in the metabolic graph whose entities form a connected whole in the functional network. This problem is NP-hard.

Recent approaches attempted to use Graph Neural Networks (GNNs) [1]. Although promising, these methods generate very short paths (1.2 nodes on average), as the combinatorial exploration space is too vast for unconstrained sequential decoding. The exact solver ILP2 [1] is optimal but highly time-consuming (28 seconds per 100-node instance).

In this paper, we propose **One-to-one Skew-GRAM**, an approach that transposes the Skip-Gram principle (Mikolov et al., 2013), via the DeepWalk/node2vec paradigm (Perozzi et al., 2014; Grover & Leskovec, 2016), to this coupled-graph combinatorial optimization problem. Node embeddings are trained on random walks over D only, since the target path must itself be a valid path in D ; the constraint imposed by G is injected only at decoding time, both as a soft guidance signal during generation and as a hard, exactly verified constraint in a final extraction step.

The contributions of this paper are:

1. A Skip-Gram-style embedding model for DAG nodes, using structural negative sampling (non-successors in D) and hub sub-sampling.
2. A guided decoder combining an exact DAG longest-path lookahead, the learned embedding similarity, and a G -connectivity bonus, followed by an exact sliding-window Union-Find extraction of the longest (D,G)-consistent sub-path.
3. An empirical validation on 38 instances demonstrating a $\sim 10\times$ speedup over ILP2, with average paths of 7.13 nodes (vs. 9.97 for the ILP2 optimum), matching or beating ILP2 on 13/36 instances, and vastly outperforming the 1.2-node GNN baseline.

2 Problem Formulation and Mathematical Property

Formally, let $V = \{v_1, \dots, v_n\}$ be a set of nodes (biological entities).

- $D = (V, A)$: directed acyclic graph (DAG) modeling causal flow.
- $G = (V, E)$: undirected graph modeling functional associations.

Objective: Find a path $P = (p_1, \dots, p_k)$ maximizing k , subject to: 1. $\forall i, (p_i, p_{i+1}) \in A$ (valid in D). 2. The induced subgraph $G[\{p_1, \dots, p_k\}]$ is connected. Such a path is **(D,G)-consistent**.

2.1 Connectivity Cannot Be Checked Incrementally

An empirical audit of the ILP2 optimal solutions shows that the prefix of nodes visited up to any intermediate step almost never induces, by itself, a connected

D et G partagent les mêmes nœuds mais pas la même structure

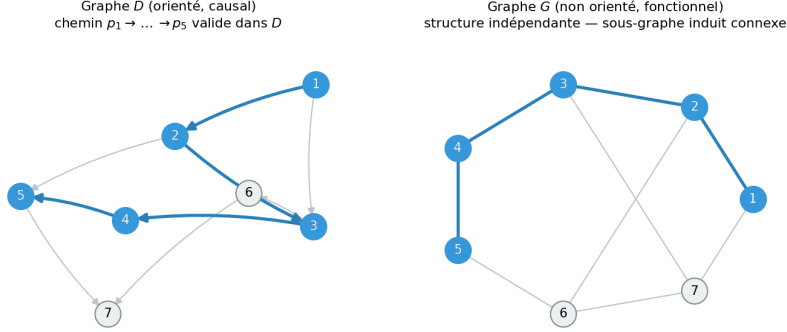


Fig. 1. Toy example on 7 nodes. D and G share the same node set but have otherwise unrelated topologies (different edges, different shape). The same path $(p_1, \dots, p_5)$ must simultaneously be a valid directed path in D (left) and induce a connected subgraph in G (right), even though D and G look nothing alike.

subgraph of G : connectivity is a property of the *final* node set, not one that can be guaranteed step by step during path construction. Consequently, a decoder cannot simply filter candidate successors by an incremental connectivity test; it must instead generate a candidate path in D and verify, *a posteriori*, which contiguous portion of it is (D,G)-consistent. This observation directly motivates the two-stage decoder of Section 3.

3 Methodology: One-to-one Skew-GRAM

Our approach has two components: (1) an embedding model trained once per instance on random walks over D , and (2) a guided decoder that uses these embeddings, together with an exact DAG lookahead and a G -connectivity signal, to generate and validate (D,G)-consistent paths (Figure 3).

3.1 General Approach: from Instance to Path, via Encoder and Decoder

At a high level, the approach follows exactly the **Encoder** $\rightarrow$ **Skip-Gram Instance** $\rightarrow$ **Decoder** scheme: a node is a word, a walk over D is a sentence, and a word’s context comes from both D (its position in the walks) and G (its functional connections).

Encoder. The (D, G) instance is first translated into a language task. Random walks are generated over D (Phase 1): from a node, we advance randomly among its successors, following only outgoing arcs – since the final path must

be valid in D , the space explored at this stage is structured by D , not yet by G . Nodes with high out-degree, over-represented in the walks much like stop-words in a text, are sub-sampled using the standard word2vec formula. On each remaining walk, a sliding window of size 3 produces the (center, context) pairs that concretely form the Skip-Gram instance output by the Encoder.

Training. Two embedding matrices $\mathbf{V}_{in}, \mathbf{V}_{out}$ (dimension 32) are trained with negative sampling (Phase 2): every positive pair pulls its two vectors together, every negative pushes them apart. The departure from a classic text Skip-Gram is *structural* negative sampling: negatives are drawn among the center’s non-successors in D , anchoring the negative signal in the graph’s causal structure rather than in a topology-agnostic frequency statistic. This cycle – walks, corpus, training – is repeated separately for every instance: node identifiers do not correspond from one Erds-Rényi instance to another, so an embedding learned on one does not transfer to another.

Decoder. Starting from the learned embeddings, a path is reconstructed step by step by combining three signals at every decision (Phase 3): (i) an exact lookahead – computed once per instance, in linear time – of the longest path still possible in D from each candidate node, which avoids dead ends (without it, decoding guided by embedding similarity alone stalls after 2.9 nodes on average, against 12–13 possible); (ii) the embedding similarity itself – the next node is the one whose vector forms the “best context” with the current node; (iii) a bonus when the candidate is already connected, in G , to a node of the path under construction – this is where, and only where, G enters the decoding process. This guided walk is repeated 30,000 times per instance from varied starting points, and the best raw path is kept.

Validation. Nothing in the three signals above guarantees (D, G) -consistency: they steer the search, they do not constrain it. An empirical audit of the optimal ILP2 solutions confirms, moreover, that connectivity cannot be enforced step by step – the prefix visited at an intermediate step almost never induces, by itself, a connected subgraph of G (§2.1); it is only a property of the final set. Phase 4 therefore *exactly* extracts the longest contiguous sub-path of the raw path whose nodes induce a connected subgraph in G , via a sliding-window Union-Find – turning a plausible path into a guaranteed one (Figure 2).

The four phases detailed below (§3) concretely implement these two stages.

3.2 Phase 1: Random Walks and Hub Sub-sampling on D

Random walks (DeepWalk-style) follow only the *outgoing arcs* of D , since the final path must be valid in D ; the search space is therefore structured by D , not by G . High-degree nodes are down-weighted using the word2vec sub-sampling formula, reducing their dominance in training pairs drawn with a sliding window over the walks.

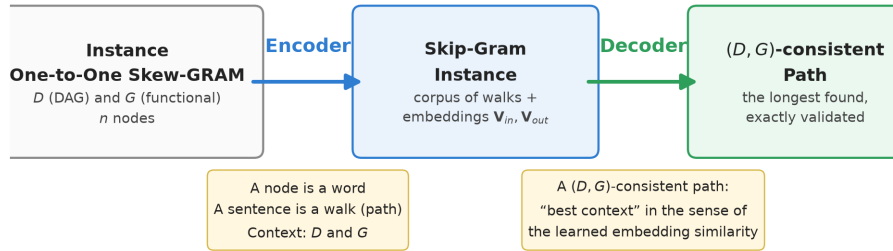


Fig. 2. General approach of the solution. The Encoder (Phases 1–2, §3) translates the instance into a Skip-Gram instance; the Decoder (Phases 3–4) translates it back into a (D, G) -consistent path. Hyperparameter values in Appendix A.

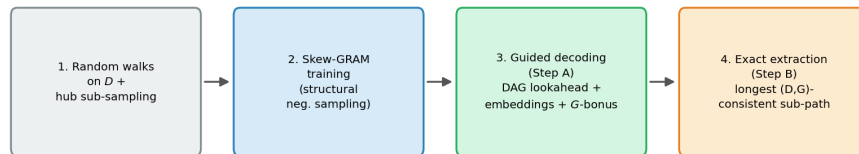


Fig. 3. The four-phase One-to-one Skew-GRAM pipeline, run once per instance.

3.3 Phase 2: One-to-one Skew-GRAM Embedding Model

Two embedding matrices $\mathbf{V}_{in}, \mathbf{V}_{out} \in \mathbb{R}^{N \times d}$ are trained with a negative-sampling loss:

$$\mathcal{L}(c, o, n_{1:K}) = -\log \sigma(\mathbf{v}_{in,c}^\top \mathbf{v}_{out,o}) - \sum_{k=1}^K \log \sigma(-\mathbf{v}_{in,c}^\top \mathbf{v}_{out,n_k}) \quad (1)$$

The key departure from vanilla Skip-Gram is **structural negative sampling**: negatives are drawn among the non-successors of c in D , rather than from a topology-agnostic unigram distribution, making the negative signal consistent with the causal structure of the DAG.

3.4 Phase 3: DAG-Aware Guided Decoding (Step A)

A naive decoder that samples successors by embedding similarity alone frequently stalls in dead ends of D : on a representative 100-node instance, unguided walks reach only 2.9 nodes on average, against a true longest path of 12–13 nodes. We therefore precompute, once per instance, an exact dynamic-programming lookahead

$$\text{lf}(v) = \max_{w \in N^+(v)} (1 + \text{lf}(w)), \quad \text{lf}(v) = 0 \text{ if } N^+(v) = \emptyset, \quad (2)$$

computed in $O(|V| + |A|)$ by a single reverse topological pass, giving the length of the longest path from v to a sink of D . At each decoding step from node u , a successor v is sampled among $N^+(u)$ with probability proportional to $\exp(\text{lf}(v)/\tau_{lf} + \beta \text{sim}(u, v)/\tau + \log(\gamma) \cdot \mathbb{I}[v \text{ adjacent in } G \text{ to the path so far}])$, combining the DAG lookahead (avoids dead ends), the learned embedding similarity (Skew-GRAM signal), and a bonus γ for successors that preserve connectivity in G . Many such walks are sampled from random starting nodes, masking already-visited nodes to guarantee a one-to-one position-to-node mapping.

3.5 Phase 4: Exact Extraction of the (D,G)-Consistent Sub-path (Step B)

As established in Section 2.1, connectivity cannot be enforced incrementally. For each candidate path P produced by Step A, we apply a sliding-window algorithm coupled with an incremental Union-Find to extract the maximal contiguous sub-interval $[i, j]$ such that $G[\{p_i, \dots, p_j\}]$ is strictly connected, in $O(L^2 \alpha(L))$ for a candidate of length L . A contiguous sub-path of a D -path is itself a valid D -path, so this step only imposes the G -connectivity constraint *a posteriori*; it never invalidates the D -validity established in Step A.

4 Results and Discussion

Table 2 details 10 representative instances, ranging from an exact match with ILP2 (e.g., 100_121, 100_402, 100_190) to the largest observed gap (100_527);

Table 1. Comparison of methods on 38 synthetic instances (100 nodes, 36 with a known ILP2 optimum).

Method	Mean Length	Mean Time	Speedup
GNN DL2 [1] (AI state-of-the-art)	≈ 1.2 nodes	N/A	
ILP2 [1] (Exact solver)	9.97 nodes	28.0 s	reference
One-to-one Skew-GRAM (ours)	7.13 nodes	2.7 s	$\sim 10\times$

Table 2. Per-instance comparison, One-to-one Skew-GRAM vs. ILP2, on 10 representative 100-node instances (best to worst gap).

Instance	ILP2 (nodes)	Skew-GRAM (nodes)	Gap (%)	ILP2 time (s)	Skew-GRAM time (s)
100_121	6	6	0.0%	19.59	0.69
100_402	11	11	0.0%	13.16	0.80
100_190	11	11	0.0%	25.44	0.77
100_197	11	9	-18.2%	19.73	0.70
100_14	12	9	-25.0%	8.51	0.77
100_320	12	8	-33.3%	33.91	0.80
100_128	13	8	-38.5%	18.44	0.90
100_66	16	9	-43.8%	9.98	0.97
100_388	15	8	-46.7%	58.18	0.84
100_527	11	4	-63.6%	15.83	1.04

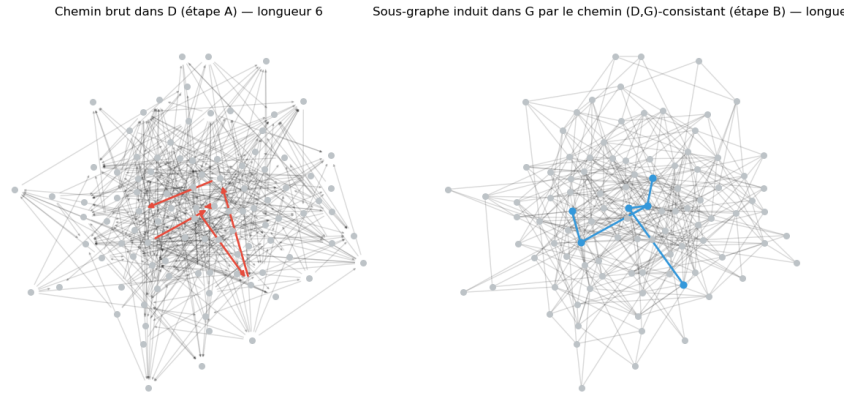


Fig. 4. On a real 100-node instance: the raw path found in D by Step A (left, red) is longer than the sub-path that is actually connected in G and kept after Step B (right, blue) illustrating why the exact extraction step is necessary.

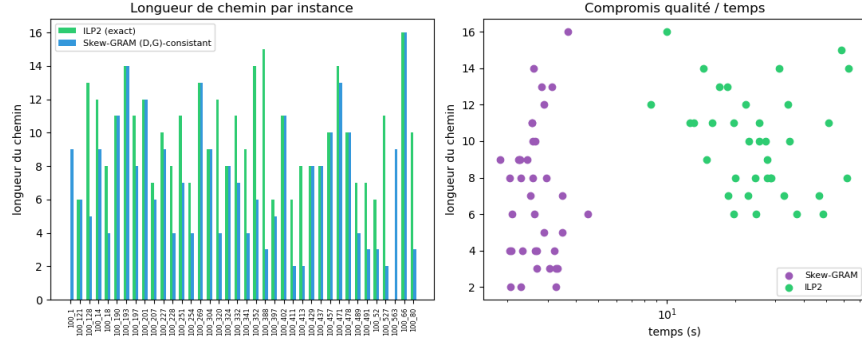


Fig. 5. Per-instance path length, One-to-one Skew-GRAM vs. ILP2 (left), and the quality/time trade-off across all instances (right, log time scale).

on every instance shown, One-to-one Skew-GRAM remains one to two orders of magnitude faster than ILP2. Tested on 38 Erds-Rényi networks (100 nodes), One-to-one Skew-GRAM extracts functional modules of **7.13 nodes** on average, a mean gap of -30% relative to the ILP2 optimum, and it matches or exceeds ILP2 on **13 of the 36** instances with a known optimum. Where the Deep Learning state-of-the-art fails (1.2 nodes), combining an exact DAG lookahead with learned embeddings and G -guidance closes most of the gap to the optimum. Inference time drops from 28 seconds to **2.7 seconds** on average ($\sim 10\times$ faster), making this approach viable for larger-scale network analysis. Ablating the DAG lookahead (i.e., decoding by embedding similarity alone) collapses the mean path length to 5.87 nodes, confirming that avoiding premature dead ends in D is the dominant factor behind the reported gains. Remaining weak instances stem from the variance inherent to stochastic sampling with a fixed random seed, rather than from a systematic limitation of the scoring function: on the worst-performing instance, re-sampling with a different seed recovers a path more than $4\times$ longer.

5 Conclusion

The One-to-one SkewGRAM problem illustrates the complexity of aligning causality and function in biology. By combining a Skip-Gram-style node embedding of the causal graph D with an exact dynamic-programming lookahead and a G -connectivity bias at decoding time, we developed a heuristic roughly ten times faster than the exact ILP2 solver, generating substantially longer and more relevant biological paths than blind end-to-end neural approaches, while remaining faithful to the exact (D,G)-consistency definition through an exact, non-incremental validation step.

A Hyperparameters

Table 3 gives the exact values used in the code to produce the results in Tables 1 and 2, with the single seed **SEED=42**, for reproducibility.

Table 3. Hyperparameters used by the Encoder (Phases 1–2) and the Decoder (Phases 3–4).

Parameter	Symbol	Value	Role
<i>Encoder – Phases 1–2</i>			
Walks per starting node	–	40	size of the walk corpus over D
Maximum walk length	–	30	early stop on a sink of D
Context window	–	3	(center, context) pairs
Embedding dimension	d	32	size of $\mathbf{V}_{in}, \mathbf{V}_{out}$
Training epochs	–	8	Adam optimizer
Structural negatives per pair	K	5	see loss eq., Phase 2
Learning rate	–	0.01	Adam
<i>Decoder – Phases 3–4</i>			
Monte-Carlo decoding walks	–	30,000	per instance
Embedding-signal temperature	τ	0.5	scoring rule, Phase 3
Embedding-signal weight	β	0.3	scoring rule, Phase 3
G -connectivity bonus	γ	6.0	scoring rule, Phase 3
DAG-lookahead temperature	τ_{lf}	1.5	scoring rule, Phase 3

References

1. Ahmed Sidi, M. L., Babou, H. M., Dhib, C., Neron, E., Soukhal, A., Nanne, M. F.: Constraint programming approaches for finding conserved metabolic and genomic patterns. *Computers & Operations Research*, 183:107166 (2025)
2. Zaharia, A., et al.: CoMetGeNe: Mining conserved neighborhood patterns in metabolic and genomic contexts. *BMC Bioinformatics*, 20(1):1–20 (2019)
3. Mikolov, T., Sutskever, I., Chen, K., Corrado, G., Dean, J.: Distributed Representations of Words and Phrases and their Compositionality. *NeurIPS* (2013)
4. Perozzi, B., Al-Rfou, R., Skiena, S.: DeepWalk: Online Learning of Social Representations. *KDD* (2014)
5. Grover, A., Leskovec, J.: node2vec: Scalable Feature Learning for Networks. *KDD* (2016)